

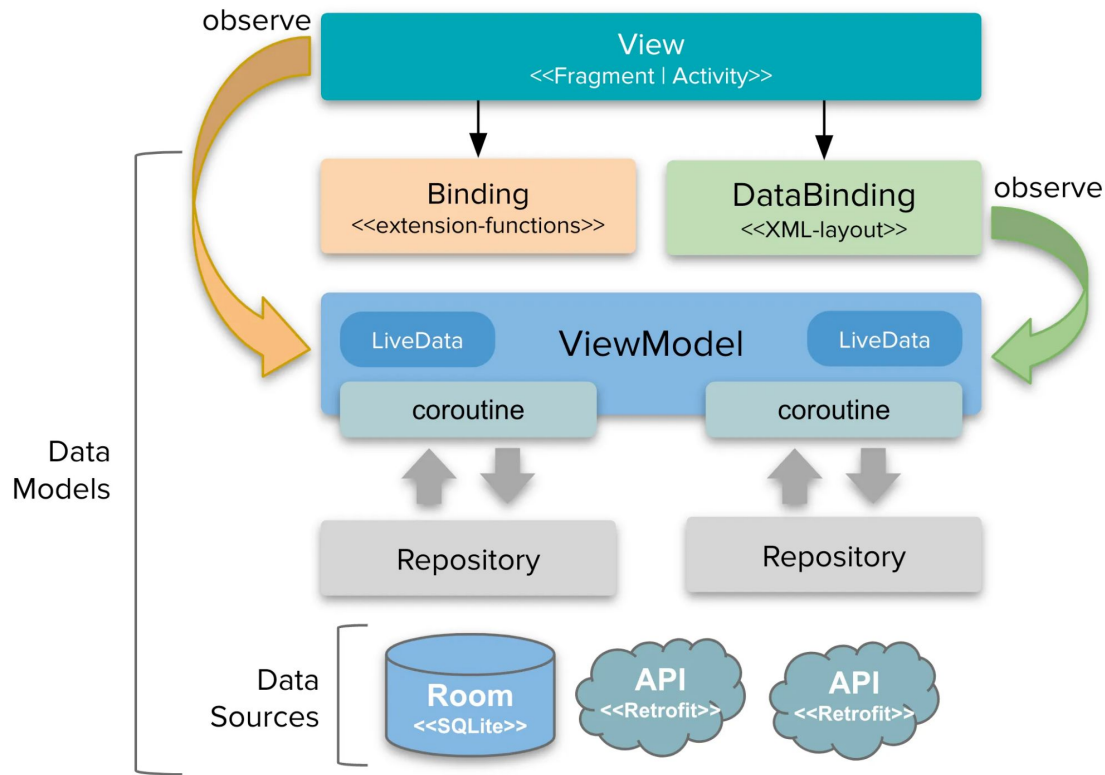
Desarrollo de Sistemas Móviles

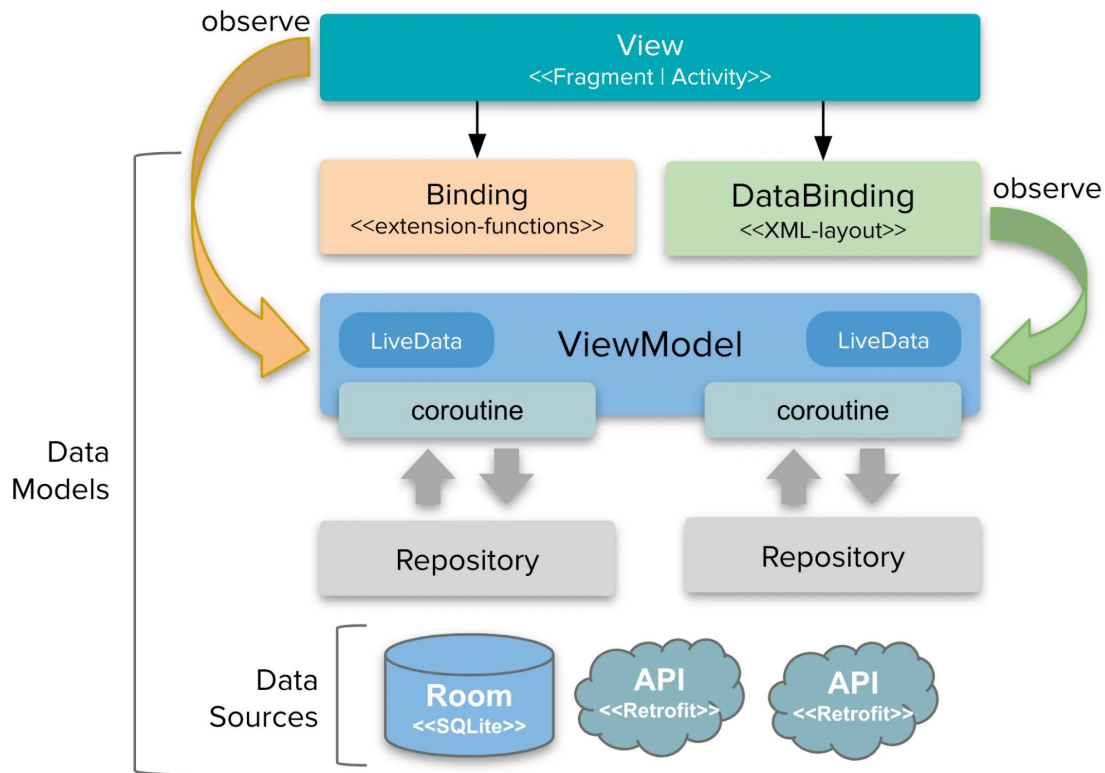
Sesión 4: Arquitectura MVVM, Navegación y Ciclo de Vida

GNSS y Perfil de Usuario

Al finalizar la clase, serás capaz de:

- Comprender la arquitectura MVVM aplicada en aplicaciones Android modernas.
- Explicar el flujo reactivo de datos usando StateFlow y UDF.
- Implementar navegación moderna utilizando Navigation Compose.
- Diferenciar LiveData y StateFlow dentro del ecosistema Android actual.
- Comprender el rol del ViewModel en el manejo del estado y lifecycle.
- Implementar concurrencia asincrónica utilizando Kotlin Coroutines.
- Entender cómo Room y Flow permiten persistencia reactiva.
- Utilizar DataStore para manejo moderno de preferencias y configuración.
- Explicar cómo Compose realiza recomposición automática basada en cambios de estado.
- Diseñar interfaces responsive utilizando herramientas de Jetpack Compose.
- Comprender la interacción entre servicios en background, persistencia y UI reactiva.
- Analizar cómo los componentes del laboratorio se integran dentro de una arquitectura desacoplada y escalable.





Explicación

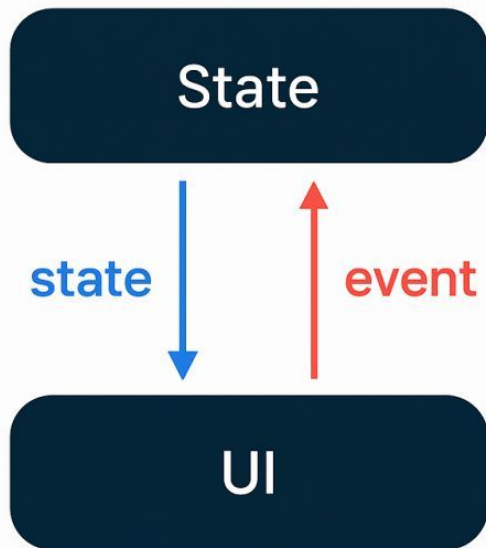
La UI nunca accede directamente a hardware o base de datos. La UI:

- observa estado
- envía eventos
- Compose recompone automáticamente.

Beneficios

- desacoplamiento
- mantenibilidad
- reactividad
- testabilidad
- escalabilidad

Unidirectional Data Flow (UDF) in Android



$$S_{t+1} = f(S_t, E_t)$$

Explicación

Cuando ocurre un evento:

- el **ViewModel** procesa lógica,
- genera un nuevo estado,
- **StateFlow** emite cambios,
- **Compose** recompone automáticamente.

Beneficios

Cuando ocurre un evento:

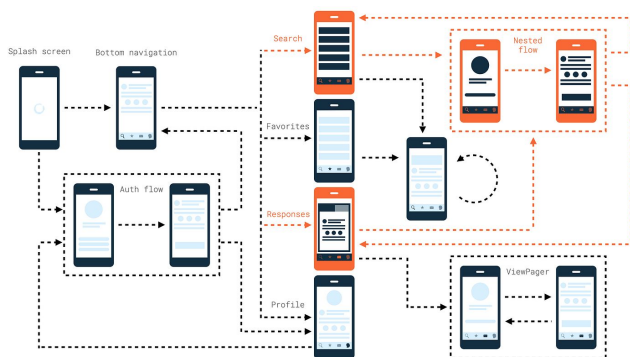
- estado predecible,
- menos race conditions,
- sincronización consistente,
- debugging más simple.

Navegación moderna

- `rememberNavController()`
- `NavHost()`
- `composable()`

Arquitectura Single-Activity

Con 1 Activity + múltiples pantallas
Compose



Explicación

- Navigation Compose centraliza navegación mediante un `NavController`.
- Las pantallas son `Composables` y no Activities independientes.

Navegación interna: Usada para manejar estados internos de `ProfileScreen`.

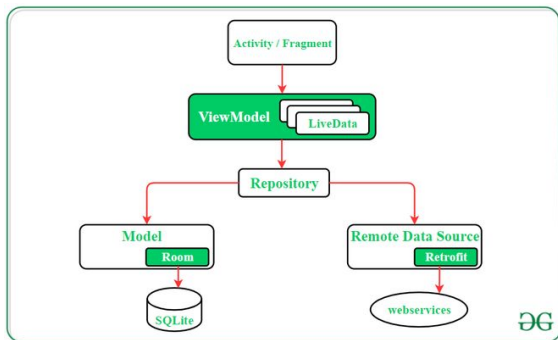
sealed class ProfileViewState

Rol del ViewModel

El ViewModel:

- almacena estado,
- ejecuta lógica,
- coordina repositorios,
- sobrevive rotaciones.

```
class GpsViewModel : ViewModel()
```



Explicación

Android destruye Activities durante cambios de configuración.

El ViewModel desacopla:

- **UI:** efímera
- **Estado:** persistente

Beneficio

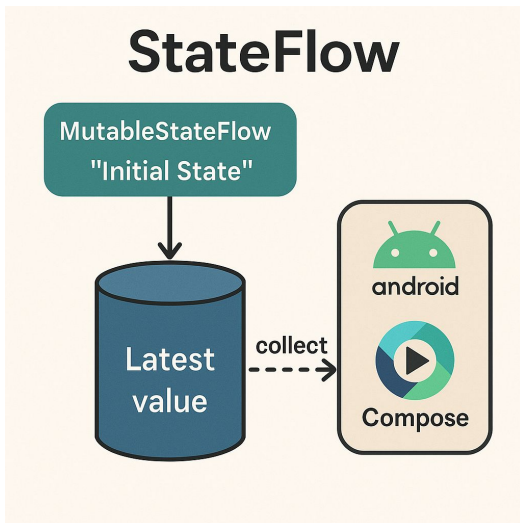
La aplicación mantiene continuidad lógica incluso cuando cambia el lifecycle visual.



¿Qué es StateFlow?

StateFlow es un stream observable y reactivo basado en Kotlin Coroutines.

```
val uiState: StateFlow<State>
```



Explicación

StateFlow mantiene siempre un valor actual observable.

Cuando cambia el estado:

- emite automáticamente
- Compose detecta cambios
- la UI se recompone

Beneficio

- sincronización automática
- concurrencia segura
- estado consistente.

```
val result = MutableLiveData  
<String()>
```



LiveData

- Life-cycle aware
- One observer
- No initial state

```
val result = MutableStateFlow  
<String('')>
```

StateFlow

- Structured concurrency
- Multiple collectors
- Initial value

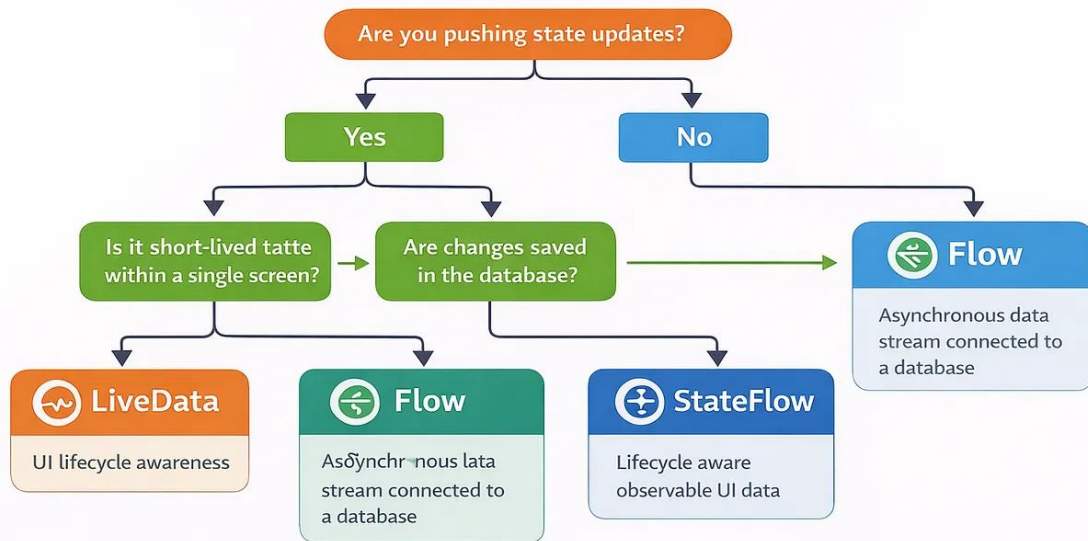


Explicación

LiveData fue diseñado para XML y arquitecturas tradicionales. **StateFlow** se integra completamente con:

- coroutines
- Flow
- Compose

LiveData vs. Flow vs. StateFlow: Decision Guide



Legend:

 **LiveData**

UI lifecycle awareness

 **Flow**

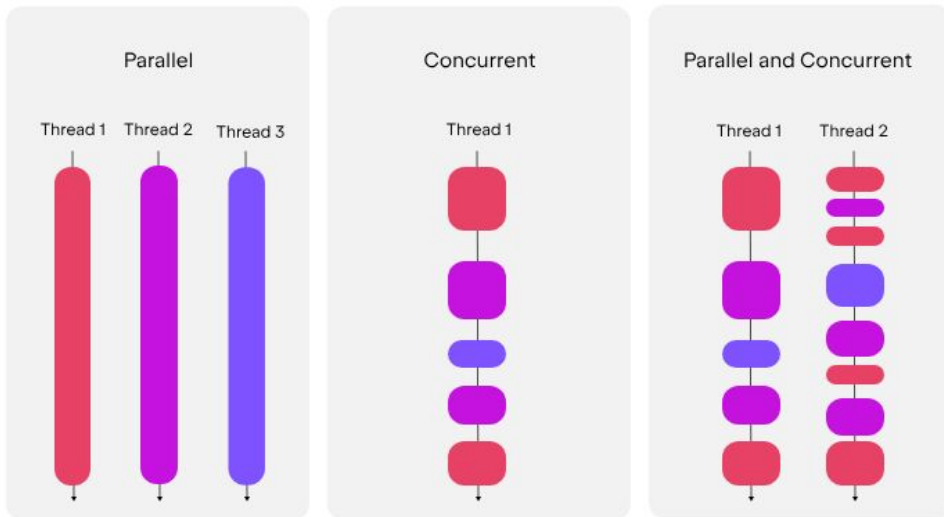
Data streams from a database or network

 **StateFlow**

Short-lived UI state within a screen

Objetivo

Manejar concurrencia asincrónica sin bloquear hilos.



Explicación

Las corrutinas permiten escribir código asincrónico como flujo secuencial.

La suspensión:

- no bloquea threads
- preserva contexto
- reduce complejidad concurrente

Operador	Función
<code>launch</code>	iniciar coroutine
<code>delay</code>	suspensión
<code>withTimeoutOrNull</code>	timeout seguro



Explicación

`combine()` sincroniza múltiples streams reactivos. Cada vez que cambia cualquiera de las fuentes:

- se reconstruye un nuevo estado global,
- la UI recibe información consistente.

Problema del laboratorio

Existen dos fuentes independientes:

- Google FLP
- Sensor GNSS

```
5
6 // combine: Emits when either flow emits (using latest values)
7 val combinedFlow = combine(priceUpdates, quantityUpdates) { price: Double, quantity: Int →
8     "Latest Price: $price, Latest Quantity: $quantity"
9 }
```

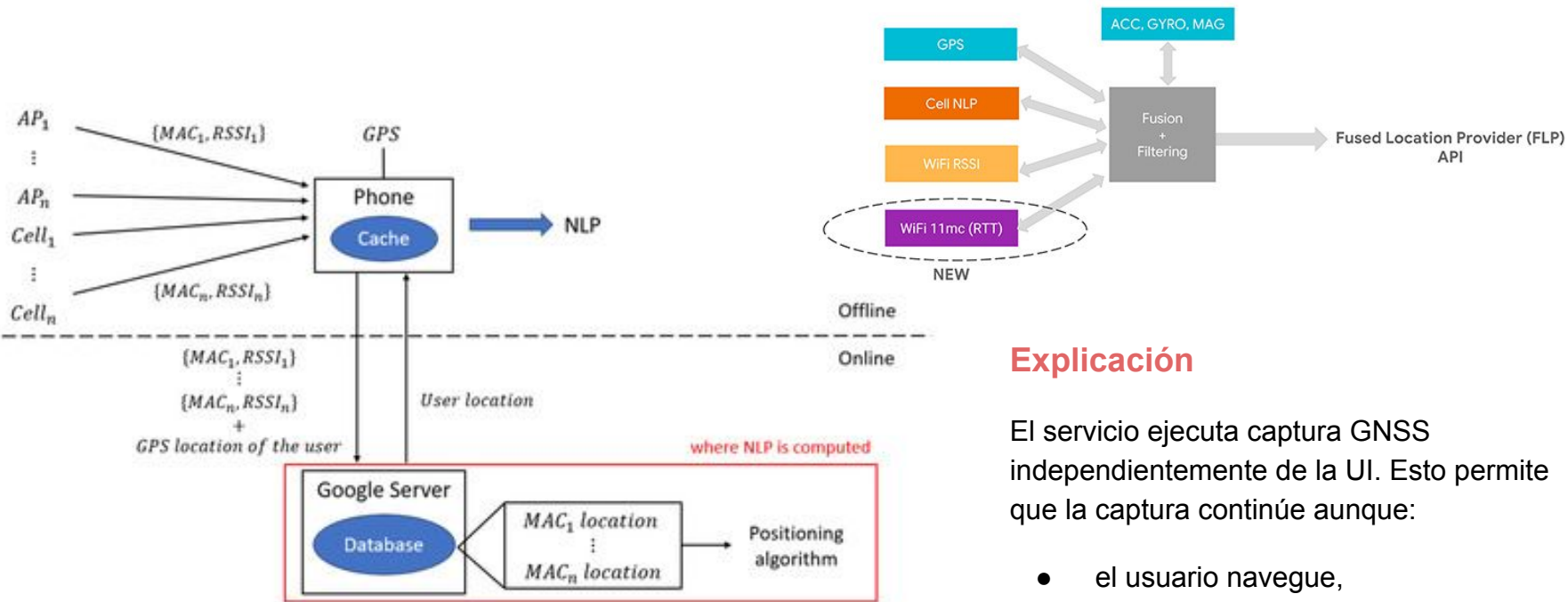


**GPS is a single satellite system
that utilizes 31 satellites**

VS



**GNSS utilizes 89 satellites
from all 4 satellite systems**



Explicación

El servicio ejecuta captura GNSS independientemente de la UI. Esto permite que la captura continúe aunque:

- el usuario navegue,
- rote pantalla,
- Compose recomponga.

Assessment of Android Network Positioning as an Alternate Source for Robust PNT



Persistencia moderna

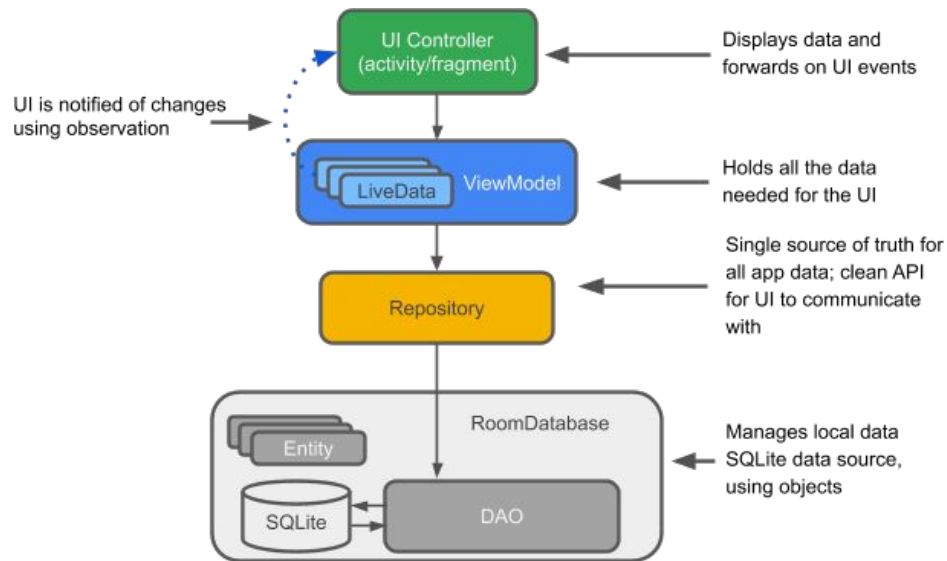
Room es el ORM oficial sobre SQLite.

Componente	Función
Entity	tabla
DAO	acceso datos
Database	contenedor

Explicación

Room integra automáticamente Flow.
Cada inserción genera nuevas emisiones reactivas.

- GpsGoogleEntity
- GpsSensorsEntity



Función del Repository

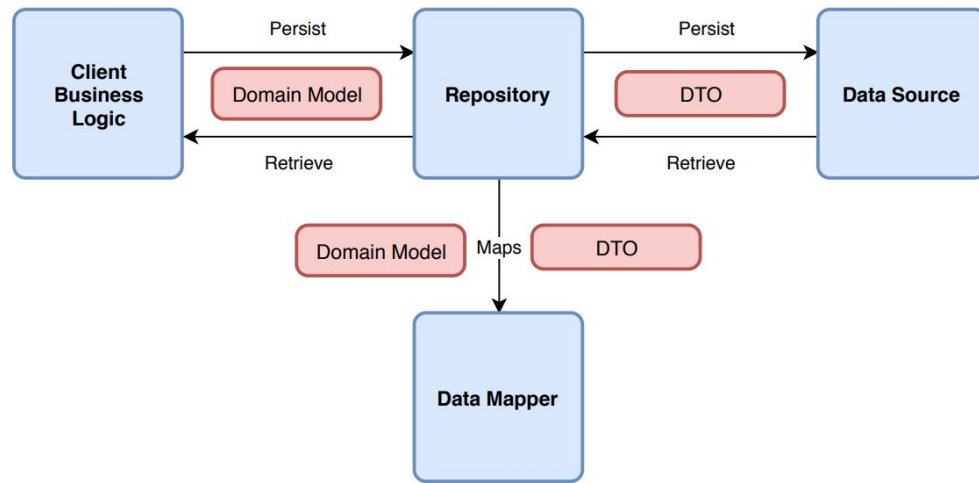
Centralizar acceso a datos.

Explicación

El Repository desacopla **ViewModel** de:

- Room,
- hardware,
- servicios,
- APIs.

La UI nunca accede directamente a DAOs.



Ventajas

- única fuente de verdad,
- menor acoplamiento,
- mejor mantenibilidad.

Persistencia moderna

DataStore reemplaza **SharedPreferences**.

Feature	SharedPreferences	DataStore
Thread-safe	✗ Not guaranteed	✓ Fully thread-safe
I/O Type	✗ Synchronous(blocking)	✓ Asynchronous (non-blocking)
Coroutines Support	✗ No	✓ Native support with Flow
Error Handling	✗ Minimal	✓ Built-in exception handling
Migration Support	Manual migration needed	✓ Migration tools available

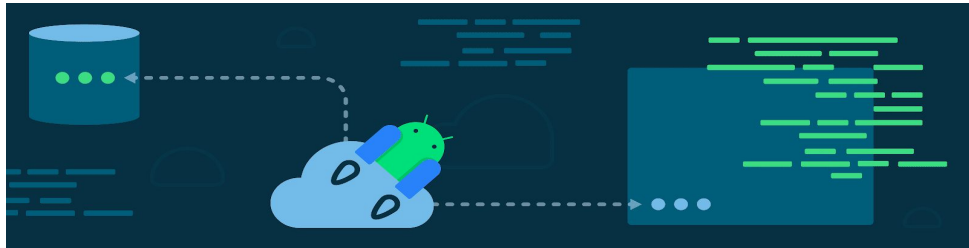
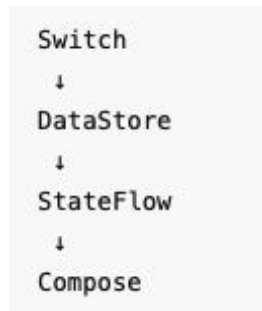
Ventajas

- asincrónico, reactivo, seguro con coroutines, basado en Flow.
- **Persistencia de:** sesión, username, dark mode.

Explicación

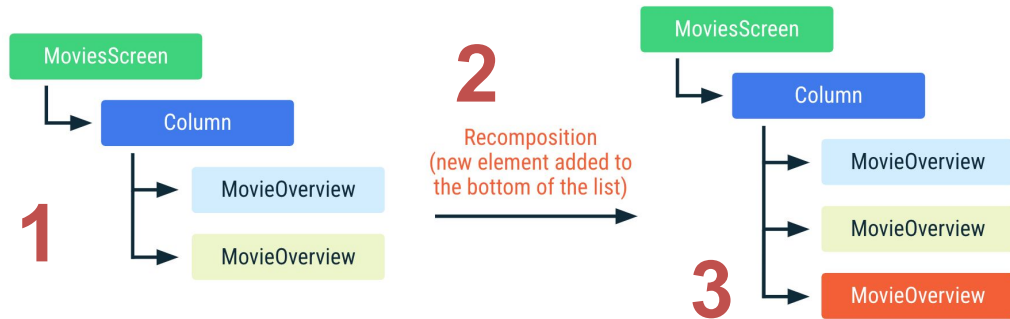
Cuando cambia el modo oscuro:

- DataStore emite,
- StateFlow actualiza,
- Compose recompone automáticamente.



UI Declarativa

Compose describe: cómo debe verse la UI según el estado actual.



1. StateFlow cambia
2. Compose detecta cambio
3. Composable se recompone

Explicación

Compose utiliza un sistema de snapshots. Cuando cambia el estado:

- invalida regiones afectadas
- recompone parcialmente
- evita rerender completo.

Beneficios

- menos boilerplate
- modularidad
- eficiencia visual

Lifecycle moderno

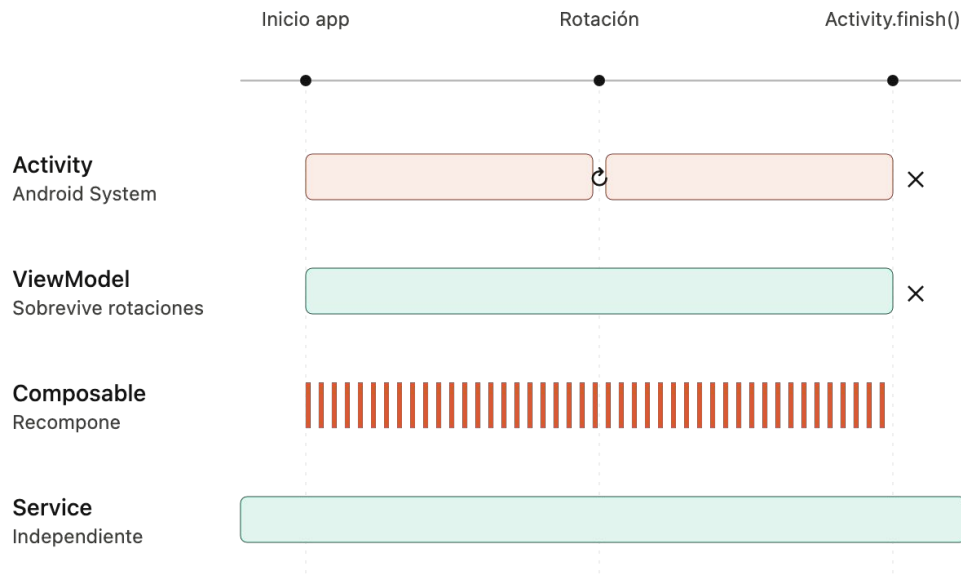
Componente	Lifecycle
Activity	Android System
ViewModel	Sobrevive rotaciones
Composable	Recompone
Service	Independiente

Explicación

onCleared() libera recursos cuando el ViewModel deja de existir:

- corrutinas
- listeners
- referencias activas

Esto evita memory leaks y procesos huérfanos.



El largo de cada barra muestra cuánto tiempo vive el componente en memoria

Objetivo

Adaptar UI a:

- tablets,
- foldables,
- landscape,
- distintos tamaños.

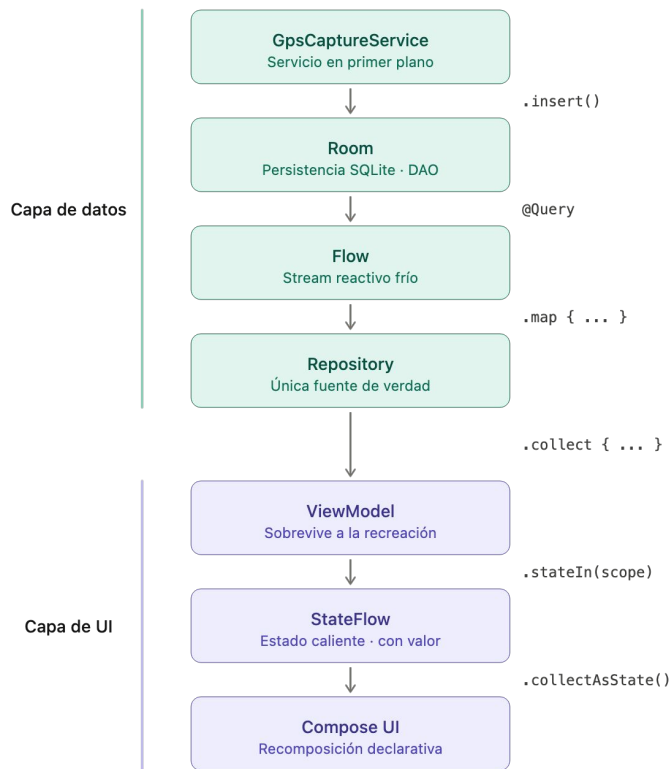
Explicación

- El layout deja de ser absoluto y pasa a ser proporcional.
- `weight()` distribuye espacio dinámicamente según tamaño disponible.

Resultado

- La interfaz mantiene consistencia visual en múltiples dispositivos.

Función/Modificador	Tipo	¿Qué hace?	Ejemplo
<code>fillMaxWidth()</code>	Modificador	Ocupa todo el ancho disponible del contenedor padre	<code>Modifier.fillMaxWidth()</code>
<code>weight()</code>	Modificador	Distribuye el espacio restante proporcionalmente (solo en Row/Column)	<code>Modifier.weight(1f)</code>
<code>LazyColumn()</code>	Composable	Lista scrollable eficiente que carga solo los elementos visibles	<code>LazyColumn { items(100) { ... } }</code>



Flujo unidireccional · reactivo · respetando el ciclo de vida

Diagrama del flujo reactivo unidireccional,
organizado en las dos capas canónicas de la
arquitectura Android moderna

(data layer y UI layer)

Tema	Aplicación
MVVM	Arquitectura
Navigation	Navigation Compose
Lifecycle	ViewModel + Service
StateFlow	Estado reactivo
Room	Persistencia local
Corrutinas	Captura GNSS
DataStore	Preferencias
Responsive UI	Compose Layouts

Lab: 11:00 AM

Lista de integrantes por grupo

Gracias

